



AI데이터분석

Contents

- **AI데이터분석 기초 프로그래밍**
 - 파이썬 프로그래밍 기초
 - 절차지향 프로그래밍
 - 파이썬 자료구조
 - 객체지향 프로그래밍
- **AI데이터분석 라이브러리**
 - 텍스트 데이터 처리
 - 넘파이 배열
 - 맷플롯립
 - 판다스
- **AI데이터분석 응용**
 - 데이터 수집
 - 데이터 전처리
 - 데이터분석과 머신러닝

3. AI데이터분석 응용

3-2. 데이터 전처리

데이터 수집

데이터 전처리

데이터 분석/응용

- 데이터 수집
 - 오픈데이터 API, 웹 크롤링, 파일 읽기, DB 액세스
- 데이터 전처리 : 넘파이, 판다스 활용
 - 데이터 클린징, 데이터 연결과 병합, 데이터 재구조화
- 데이터 분석/응용
 - 데이터 시각화(맷플롯립 활용), 기계학습

- 편향 없이 명확하고 깨끗한 데이터를 확보하는 작업
- 데이터 클리닝 작업
 - 결측 데이터 처리
 - 결측 데이터 확인
 - 결측 데이터 대체/제거 [평균 대체법]
 - 결측 데이터 반영 확인
 - 이상 데이터 처리
 - 이상 데이터 확인
 - 이상 데이터 대체/제거
 - 이상 데이터 처리 확인
 - 중복 데이터 처리
 - 중복 데이터 확인
 - 중복 데이터 처리(유일한 1개 키만 남기고 나머지 중복 제거)

- 결측값(Missing data)
 - 데이터 누락값
- 결측값이 있는 상태로 분석하게 되면 변수간의 관계가 왜곡
 - 분석의 정확성이 떨어짐
- 판다스에서는 결측값을 'NaN(Not a Number)' 으로 표기
 - 'None'도 결측값을 의미
- 결측값을 처리 방법
 - 제거(Deletion)
 - 결측값을 포함한 행, 열을 삭제하는 것
 - 대체(Imputation)
 - 결측값을 다른 값(예, 대표값인 평균)으로 변환하는 방법

- 판다스 `isnull()` 메소드 = `isna()` 메소드
 - 결측 데이터이면 `True`값을 반환
 - 유효한 데이터가 존재하면 `False`를 반환
- 판다스 `notnull()` 메소드
 - 유효한 데이터가 존재하면 `True`를 반환
 - 누락 데이터면 `False`를 반환

	Pregnancies	BloodPressure	BMI	Age	Outcome
0	6.0	NaN	33.6	50	1.0
1	NaN	66.0	NaN	31	NaN
2	8.0	64.0	23.3	21	1.0
3	NaN	NaN	28.1	40	NaN
4	0.0	40.0	NaN	33	1.0

`df.isnull()`

	Pregnancies	BloodPressure	BMI	Age	Outcome
0	False	True	False	False	False
1	True	False	True	False	True
2	False	False	False	False	False
3	True	True	False	False	True
4	False	False	True	False	False

`df.notnull()`

	Pregnancies	BloodPressure	BMI	Age	Outcome
0	True	False	True	True	True
1	False	True	False	True	False
2	True	True	True	True	True
3	False	False	True	True	False
4	True	True	False	True	True

- isna() 메소드 = isnull() 메소드

```
import pandas as pd
import matplotlib.pyplot as plt
import datetime as dt

weather = pd.read_csv('https://raw.githubusercontent.com/dongupak/DataSciPy/master/data/csv/weather.csv',
                      encoding='CP949')

weather[ weather['평균풍속'].isna() == True ]
```

	일시	평균기온	최대풍속	평균풍속
559	2012-02-11	-0.7	NaN	NaN
560	2012-02-12	0.4	NaN	NaN
561	2012-02-13	4.0	NaN	NaN
1694	2015-03-22	10.1	11.6	NaN
1704	2015-04-01	7.3	12.1	NaN
3182	2019-04-18	15.7	11.7	NaN

- 판다스 결측치 개수 확인 함수
 - 칼럼별(행방향) 결측값 개수 구하기
 - `df.isnull().sum() == df.isnull().sum(0)`
 - 행(row) 단위(열방향)로 결측값 개수 구하기
 - `df.isnull().sum(1)`
 - 행(row) 단위(열방향)로 실제값 개수 구하기
 - `df.notnull().sum(1)`

- 결측치 제거 메소드
 - 행 삭제
 - `df.dropna(axis=0)`
 - 열 삭제
 - `df.dropna(axis=1)`
 - `df.dropna()`
- 결측치 대체 메소드
 - 결측값을 특정 값으로 채우기
 - `df.fillna(0)`
 - 결측값을 특정 문자열로 채우기
 - `df.fillna('')`
 - 결측값을 변수별 평균으로 대체
 - `df.fillna(df.mean())`

- `weather.dropna(axis=0, how="any", inplace=True)`
 - `axis=0` → 결측치를 포함하는 행 삭제
 - `how="any"` → 하나의 칼럼이라도 NaN이면 행 삭제
 - `inplace=True` → 현재 데이터프레임에 변경 적용

```
import pandas as pd
import matplotlib.pyplot as plt
import datetime as dt

weather = pd.read_csv('https://raw.githubusercontent.com/dongupak/DataSciPy/master/data/csv/weather.csv',
                      encoding='CP949')

weather.dropna(axis=0, how="any", inplace=True)

weather[ weather['평균풍속'].isna() == True ]
```

일시 평균기온 최대풍속 평균풍속

- `weather.fillna(0, inplace=True)`
 - `0` → 결측치를 0으로 대체
 - `inplace=True` → 현재 데이터프레임에 변경 적용

```
import pandas as pd

weather = pd.read_csv('https://raw.githubusercontent.com/dongupak/DataSciPy/master/data/csv/weather.csv',
                      encoding='CP949')

weather.fillna(0, inplace = True)
weather[ weather['평균풍속'] ==0.0]
```

	일시	평균기온	최대풍속	평균풍속
559	2012-02-11	-0.7	0.0	0.0
560	2012-02-12	0.4	0.0	0.0
561	2012-02-13	4.0	0.0	0.0
1694	2015-03-22	10.1	11.6	0.0
1704	2015-04-01	7.3	12.1	0.0
3182	2019-04-18	15.7	11.7	0.0

- 평균값으로 결측치를 대체

- `weather['최대풍속'].fillna(weather['최대풍속'].mean(), inplace=True)`
- `weather['평균풍속'].fillna(weather['평균풍속'].mean(), inplace=True)`

```
import pandas as pd

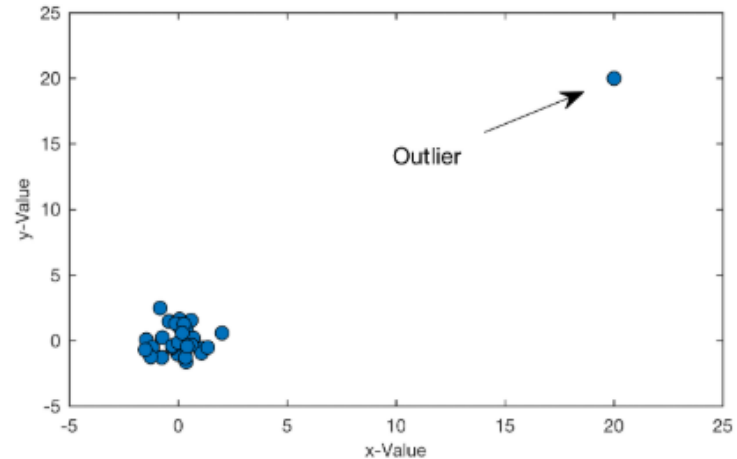
weather = pd.read_csv('https://raw.githubusercontent.com/dongupak/DataSciPy/master/data/csv/weather.csv',
                      encoding='CP949')

weather['최대풍속'].fillna(weather['최대풍속'].mean(), inplace=True)
weather['평균풍속'].fillna(weather['평균풍속'].mean(), inplace=True)

weather[weather['일시'] == '2012-02-11']
```

	일시	평균기온	최대풍속	평균풍속
559	2012-02-11	-0.7	7.911099	3.936441

- 정상에서 벗어난 데이터를 이상치(Outlier)
- 이상탐지(Anomaly Detection)
 - 이상한(비정상적인) 데이터를 검출하는 것
- 이상데이터 처리 방법
 - 이상데이터 확인
 - 이상데이터 결측 대체/제거
 - 이상데이터 반영 확인



- 박스(Box)

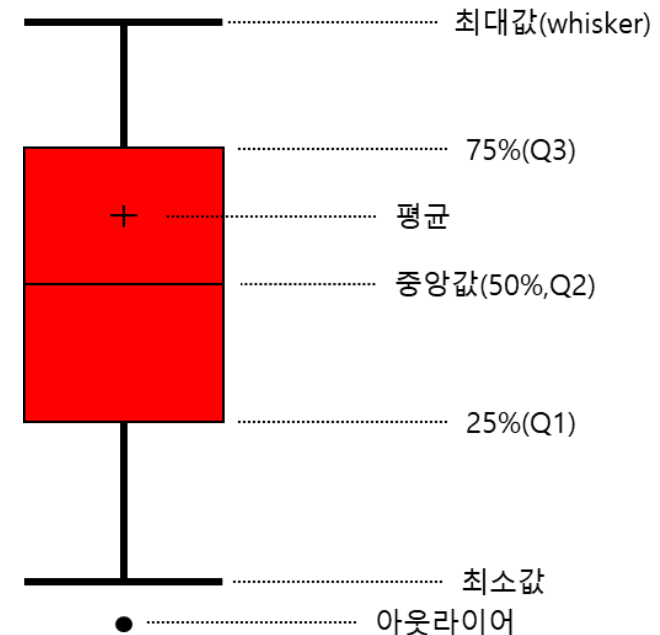
- 25%(Q1) ~75%(Q3) 까지 값

- 수염 (whiskers)

- 박스의 각 모서리(Q1, Q3)로 부터 1.5배 내에 있는 가장 멀리 떨어진 데이터 점까지 이어져 있는 것

- 이상치(Outlier)

- 수염(whiskers)보다 바깥쪽 데이터



출처 : <https://goodtcreate.tistory.com/>

- 단순 삭제
 - 이상값이 논리적 에러에 의해서 발생한 경우 해당 관측치를 삭제
 - 단순 오타나, 주관식 설문 등의 비현실적인 응답, 데이터 처리 과정에서의 오류 등의 경우에 사용
- 다른 값으로 대체
 - 데이터의 개수가 작은 경우
 - 관측치를 삭제하는 대신 다른 값(평균 등)으로 대체
- 변수화
 - 자연발생적인 이상값의 경우
- 리샘플링
 - 해당 이상값을 분리해서 모델을 만드는 방법
- 케이스분리 분석
 - 자연 발생한 이상값에 별다른 특이점이 발견되지 않는다면, 단순 제외 보다는 분석 케이스를 분리하여 분석

- 데이터를 수집하는 과정 중 또는 데이터를 병합하는 단계에서 오류 등으로 인해 데이터가 중복 발생
- 특히, 유일한 키(key) 값을 관리해야 하는 경우 중복(Duplicates)데이터가 발생하면 분석에 영향을 끼칠 수 있음
- 데이터 분석 전에 중복 데이터를 확인하고 처리하는 데이터 클리징 작업 필요
- 판다스 중복 데이터 처리 메소드
 - 중복 데이터 확인
 - `df.duplicated()`
 - 중복값 삭제
 - `df.drop_duplicates()`

- 중복이 있으면 처음이나 끝에 무엇을 남길지 확인
- `df.duplicated([칼럼명], keep='first')`
 - `keep= ' first '` → default
 - 중복값이 있으면 첫번째 값을 duplicated 여부를 False로 반환
 - 나머지 중복값에 대해서는 True를 반환
 - `keep='last'`
 - `keep=False`
 - 처음이나 끝값인지 여부는 고려를 안하고 중복이면 무조건 True를 반환

```
import pandas as pd

data = {'key1': ['a', 'b', 'b', 'c', 'c'],
        'key2': ['v', 'w', 'w', 'x', 'y'],
        'col': [1, 2, 3, 4, 5]}

df = pd.DataFrame(data, columns=['key1', 'key2', 'col'])

print(df.duplicated(['key1']))
print('-----')
print(df.duplicated(['key1', 'key2']))
```

```
0    False
1    False
2     True
3    False
4     True
dtype: bool
```

```
-----
0    False
1    False
2     True
3    False
4    False
dtype: bool
```

```
import pandas as pd

data = {'key1': ['a', 'b', 'b', 'c', 'c'],
        'key2': ['v', 'w', 'w', 'x', 'y'],
        'col': [1, 2, 3, 4, 5]}

df = pd.DataFrame(data, columns=['key1', 'key2', 'col'])

print(df.duplicated(['key1']))

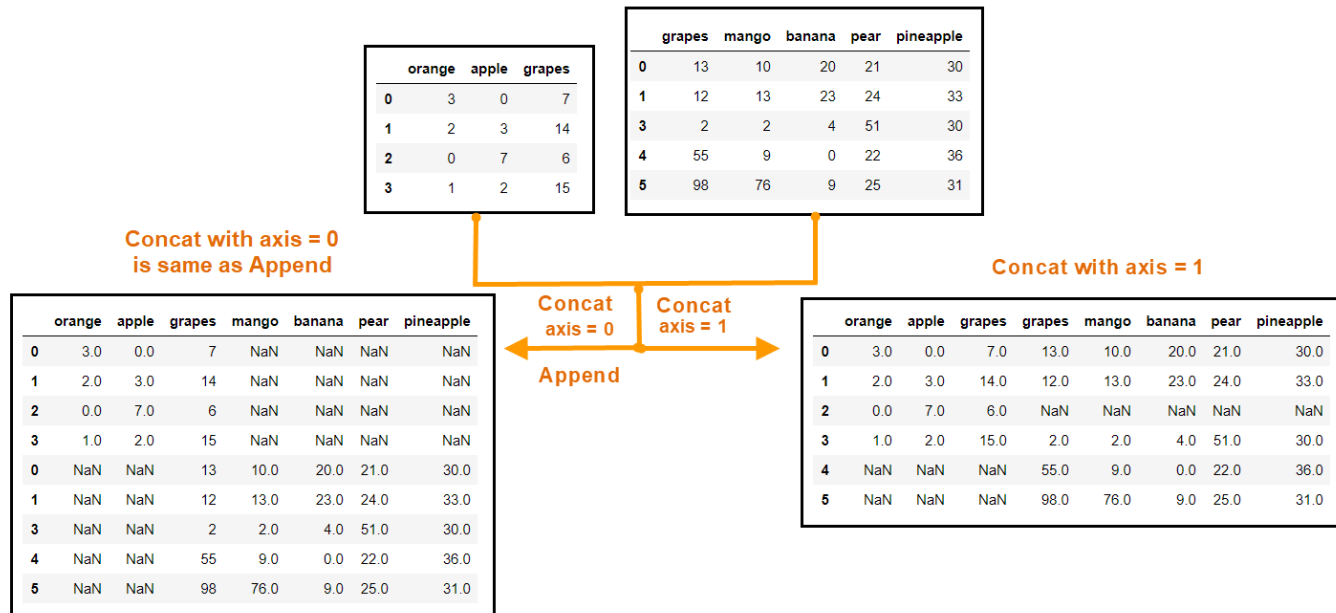
df.drop_duplicates(['key1'], keep='first')
```

```
0    False
1    False
2     True
3    False
4     True
dtype: bool
```

	key1	key2	col
0	a	v	1
1	b	w	2
3	c	x	4

데이터 연결(concatenate)

- 데이터프레임을 행과 열로 위/아래로 결합하는 방법
- 인덱스 값 중복에 주의 필요
- 판다스 데이터프레임 연결 메소드
 - `df.concat()`
 - `df.append()`



출처 : <https://khalidpark2029.tistory.com/7>

- `pd.concat(df_list, axis=0, join='outer')` 함수
 - `df_list` → 합칠 데이터프레임의 리스트
 - `axis = 0` → 행방향 연결(행 확장), **칼럼 조인**
 - `join='outer'` → 칼럼들의 합집합 (cf. inner : 칼럼들의 교집합)

```
import pandas as pd

df_1 = pd.DataFrame( {'A' : ['a10', 'a11', 'a12'],
                      'B' : ['b10', 'b11', 'b12'],
                      'C' : ['c10', 'c11', 'c12']} , index = ['가', '나', '다'] )

df_2 = pd.DataFrame( {'B' : ['b23', 'b24', 'b25'],
                      'C' : ['c23', 'c24', 'c25'],
                      'D' : ['d23', 'd24', 'd25']} , index = ['다', '라', '마'] )

df_3 = pd.concat( [df_1, df_2])
df_3
```

	A	B	C	D
가	a10	b10	c10	NaN
나	a11	b11	c11	NaN
다	a12	b12	c12	NaN
다	NaN	b23	c23	d23
라	NaN	b24	c24	d24
마	NaN	b25	c25	d25



	A	B	C
가	a10	b10	c10
나	a11	b11	c11
다	a12	b12	c12

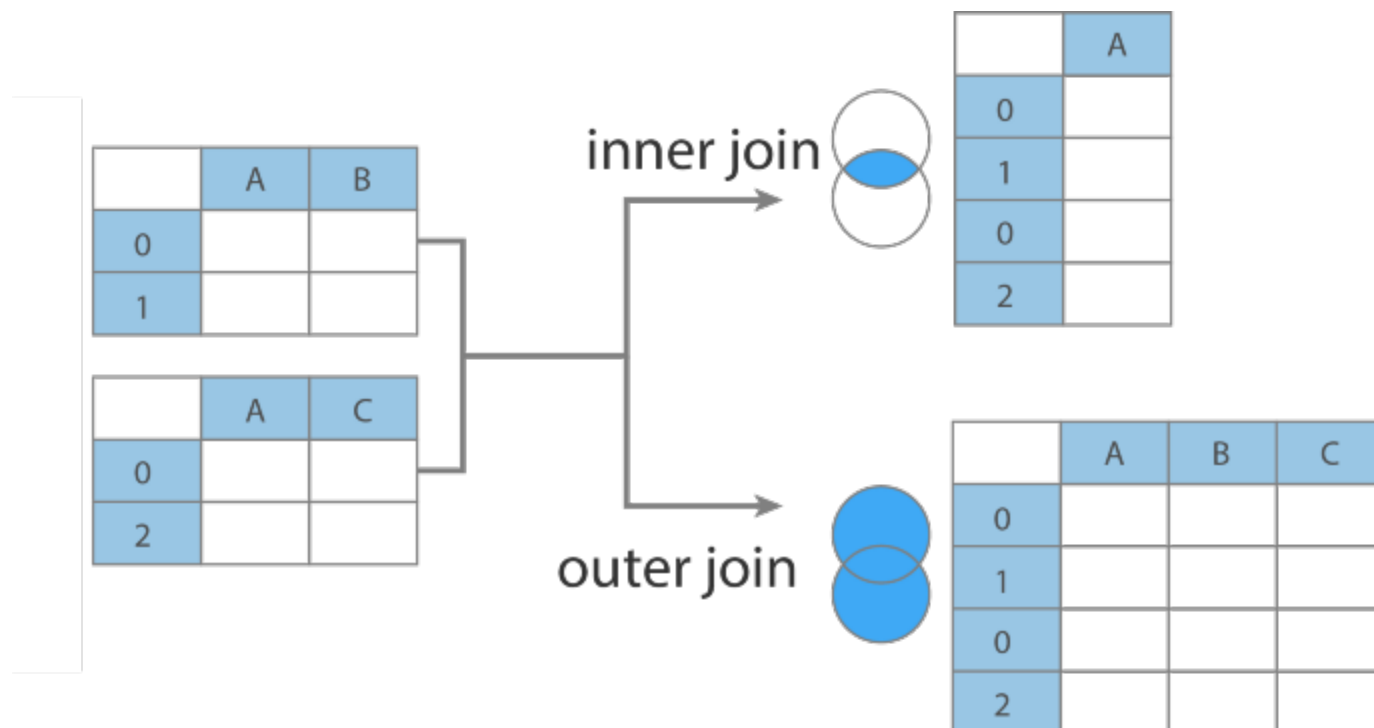
인덱스

	B	C	D
다	b23	c23	d23
라	b24	c24	d24
마	b25	c25	d25

인덱스

실습. concat()함수 outer 조인

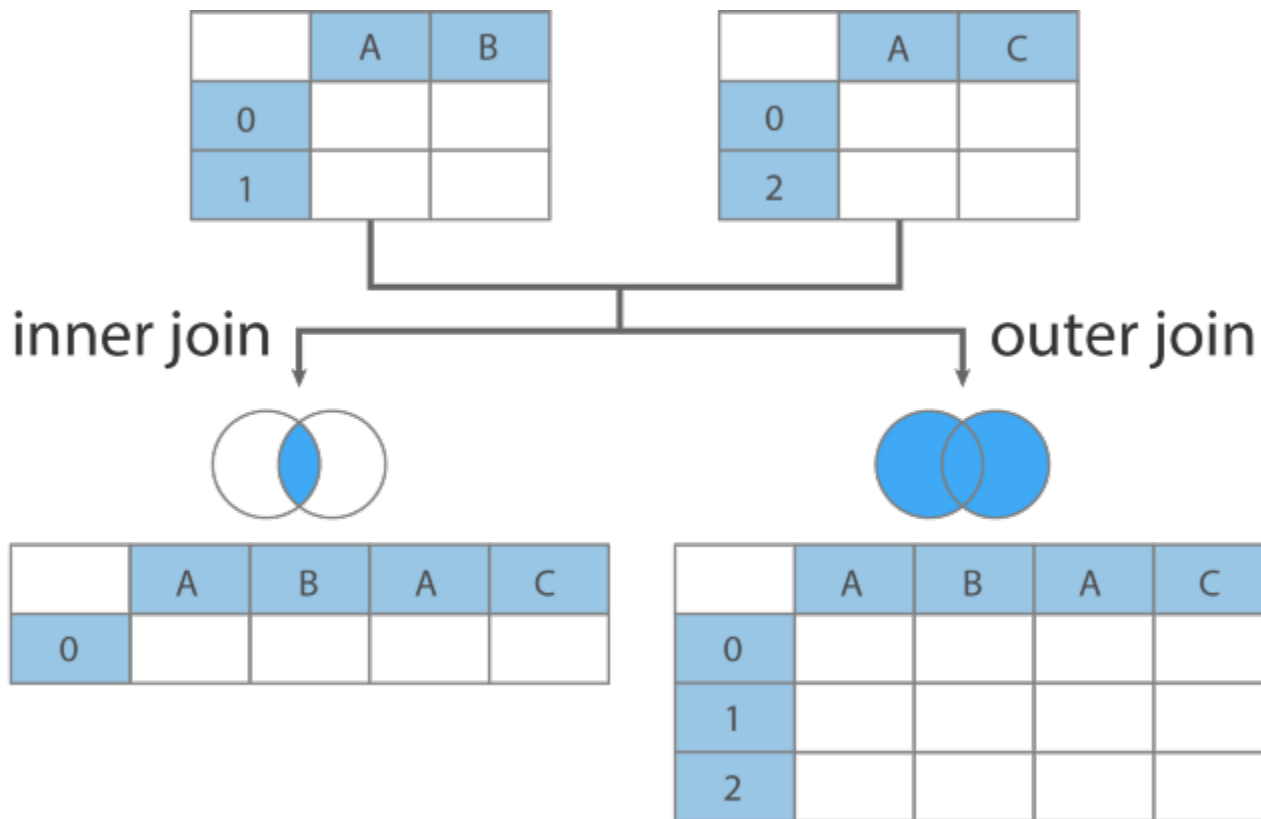
- axis=0일 때, 행 확장, 칼럼 조인



출처 : <https://ichi.pro/ko/python-pandas-dataframe-join-byeonghab-mich-yeongyeol-145789916147576>

실습. concat() 함수 outer 조인

- axis=1일 때, 열 확장, 인덱스 조인



출처 : <https://ichi.pro/ko/python-pandas-dataframe-join-byonghab-mich-yeongyeol-145789916147576>

실습. concat()함수 outer 조인

```
import pandas as pd

df_1 = pd.DataFrame( {'A' : ['a10', 'a11', 'a12'],
                      'B' : ['b10', 'b11', 'b12'],
                      'C' : ['c10', 'c11', 'c12']} , index = ['가', '나', '다'] )

df_2 = pd.DataFrame( {'B' : ['b23', 'b24', 'b25'],
                      'C' : ['c23', 'c24', 'c25'],
                      'D' : ['d23', 'd24', 'd25']} , index = ['다', '라', '마'] )

print( pd.concat( [df_1, df_2] , axis = 0, join = 'outer' ) )
print('-----')
print( pd.concat( [df_1, df_2] , axis = 1, join = 'outer' ) )
```

	A	B	C	D
가	a10	b10	c10	NaN
나	a11	b11	c11	NaN
다	a12	b12	c12	NaN
다	NaN	b23	c23	d23
라	NaN	b24	c24	d24
마	NaN	b25	c25	d25

	A	B	C	B	C	D
가	a10	b10	c10	NaN	NaN	NaN
나	a11	b11	c11	NaN	NaN	NaN
다	a12	b12	c12	b23	c23	d23
라	NaN	NaN	NaN	b24	c24	d24
마	NaN	NaN	NaN	b25	c25	d25

실습. concat()함수 inner 조인

```
import pandas as pd

df_1 = pd.DataFrame( {'A' : ['a10', 'a11', 'a12'],
                      'B' : ['b10', 'b11', 'b12'],
                      'C' : ['c10', 'c11', 'c12']} , index = ['가', '나', '다'] )

df_2 = pd.DataFrame( {'B' : ['b23', 'b24', 'b25'],
                      'C' : ['c23', 'c24', 'c25'],
                      'D' : ['d23', 'd24', 'd25']} , index = ['다', '라', '마'] )

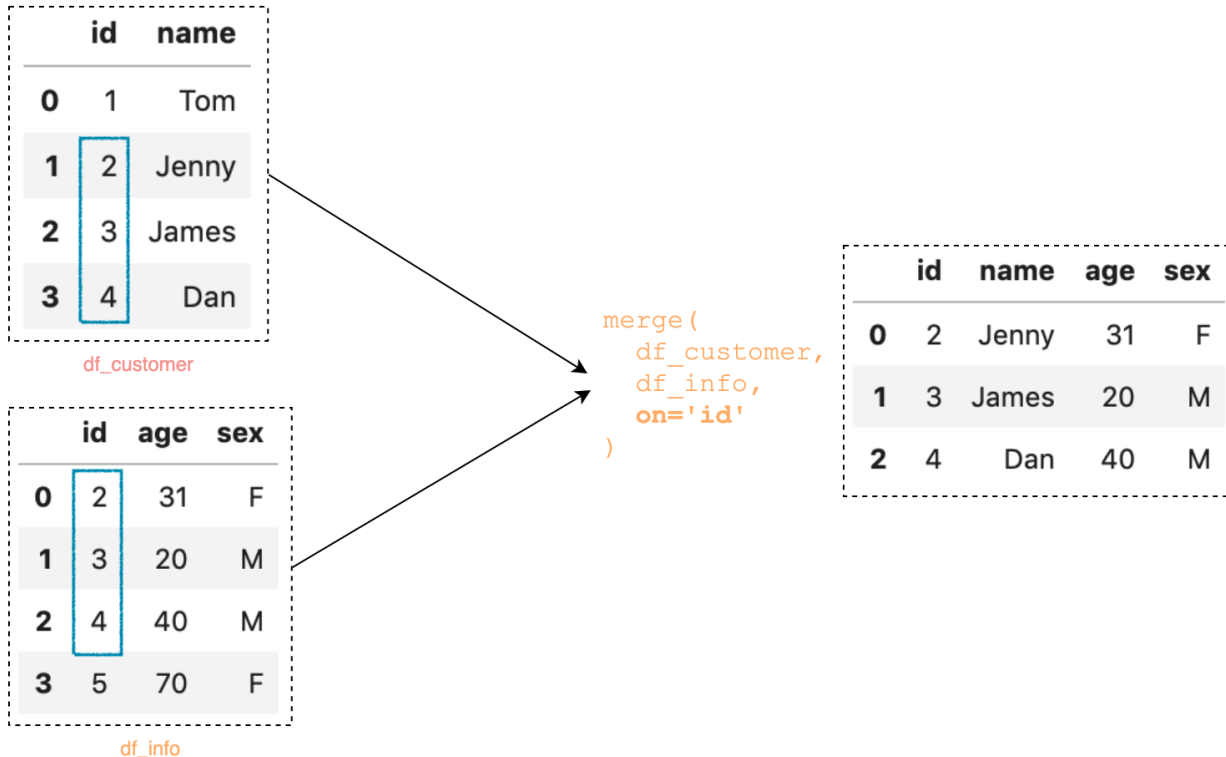
print( pd.concat( [df_1, df_2] , axis = 0, join = 'inner' ) )
print('-----')
print( pd.concat( [df_1, df_2] , axis = 1, join = 'inner' ) )
```

	B	C
가	b10	c10
나	b11	c11
다	b12	c12
다	b23	c23
라	b24	c24
마	b25	c25

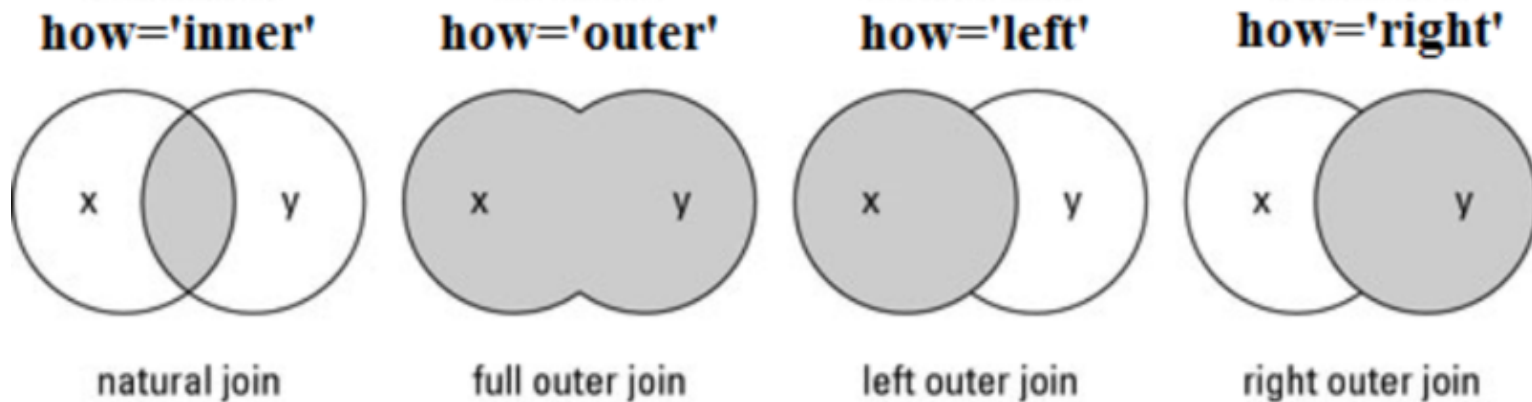
	A	B	C	B	C	D
다	a12	b12	c12	b23	c23	d23

데이터 병합(merge)

- 데이터베이스 조인 연산과 유사 방식
- 두 데이터 프레임의 공통 열 혹은 인덱스를 기준으로 두 개의 테이블을 병합하는 과정
- 이 때 기준이 되는 열, 행의 데이터를 키(key)라고 함



- 판다스 데이터 병합 지원 메소드
 - `df1.merge(df2, how='inner', on=None)`
 - 데이터프레임 `df_1`을 `df_2`와 병합
 - `how` → 결합 방식
 - left, right, inner, outer(full)
 - `on` → 조인 연산 수행되는 **열이름**



출처 : https://m.blog.naver.com/masterwu/221744949958?view=img_6

실습. 데이터병합(4가지 결합방식)

```
import pandas as pd

df_1 = pd.DataFrame( {'A' : ['a10', 'a11', 'a12'],
                      'B' : ['b10', 'b11', 'b12'],
                      'C' : ['c10', 'c11', 'c12']} , index = ['가', '나', '다'] )

df_2 = pd.DataFrame( {'B' : ['b23', 'b24', 'b25'],
                      'C' : ['c23', 'c24', 'c25'],
                      'D' : ['d23', 'd24', 'd25']} , index = ['다', '라', '마'] )

print('left outer \n' , df_1.merge(df_2, how='left', on='B' ) )
print('right outer \n' ,df_1.merge(df_2, how='right', on='B' ) )
print('full outer \n' ,df_1.merge(df_2, how='outer', on='B' ) )
print('inner \n' ,df_1.merge(df_2, how='inner', on='B' ) )|
```

left outer

	A	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN

right outer

	A	B	C_x	C_y	D
0	NaN	b23	NaN	c23	d23
1	NaN	b24	NaN	c24	d24
2	NaN	b25	NaN	c25	d25

full outer

	A	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN
3	NaN	b23	NaN	c23	d23
4	NaN	b24	NaN	c24	d24
5	NaN	b25	NaN	c25	d25

inner

Empty DataFrame
Columns: [A, B, C_x, C_y, D]
Index: []

실습. 데이터병합 (인덱스를 키로 활용)

```
import pandas as pd

df_1 = pd.DataFrame( {'A' : ['a10', 'a11', 'a12'],
                      'B' : ['b10', 'b11', 'b12'],
                      'C' : ['c10', 'c11', 'c12']} , index = ['가', '나', '다'] )

df_2 = pd.DataFrame( {'B' : ['b23', 'b24', 'b25'],
                      'C' : ['c23', 'c24', 'c25'],
                      'D' : ['d23', 'd24', 'd25']} , index = ['다', '라', '마'] )

df_3 = df_1.merge(df_2, how = 'outer', left_index = True, right_index = True )
df_3
```

	A	B_x	C_x	B_y	C_y	D
가	a10	b10	c10	NaN	NaN	NaN
나	a11	b11	c11	NaN	NaN	NaN
다	a12	b12	c12	b23	c23	d23
라	NaN	NaN	NaN	b24	c24	d24
마	NaN	NaN	NaN	b25	c25	d25

- 분석 대상 변수를 그룹별로 데이터를 집계하여 진행하는 분석
- 특정한 조건에 맞는 데이터가 하나 이상 데이터 그룹을 이루는 경우 자주 사용
- 그룹 분석은 대부분 '범주형 변수'가 그룹 연산의 기준
- 일반적으로 범주형과 연속형이 결합된 경우 그룹 분석 진행

	student_no	class	science	english	math	sex
0	1	A	50	98	50	m
1	2	A	60	97	60	w
2	3	A	78	86	45	w
3	4	A	58	98	30	m
4	5	B	65	80	90	w
5	6	B	98	89	50	m
6	7	B	45	90	80	m
7	8	B	25	78	90	w
8	9	C	15	98	20	w
9	10	C	45	93	50	w

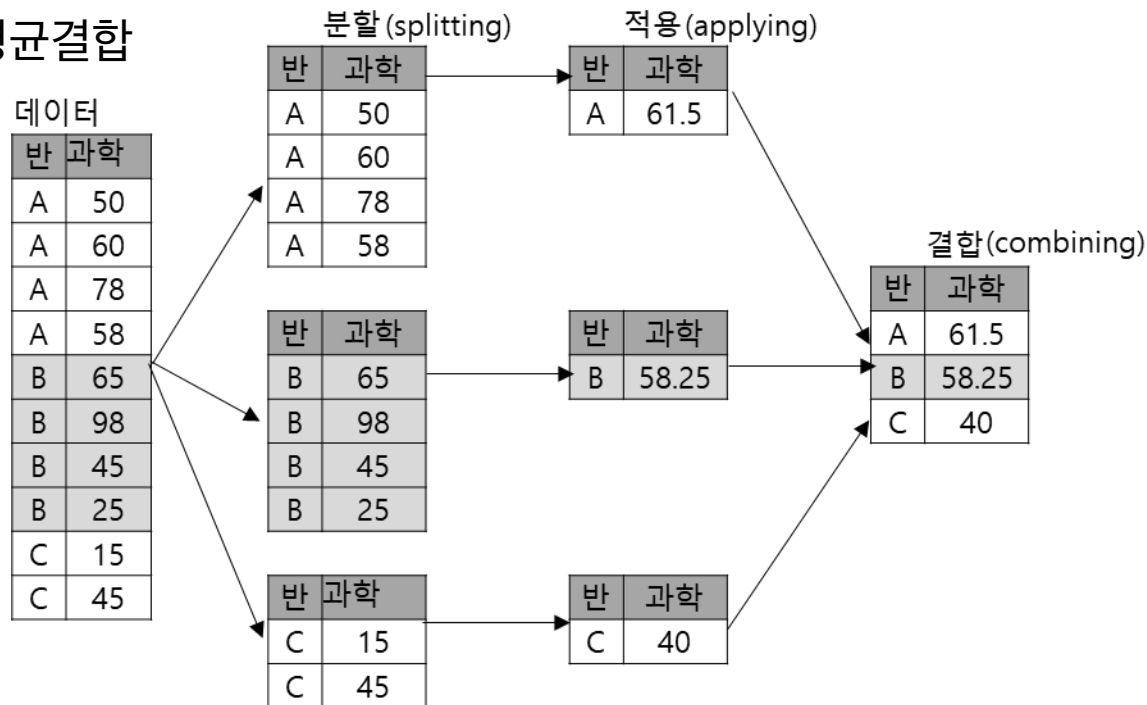
반별 그룹화



`df.groupby(['class'])`

	student_no	class	science	english	math	sex
0	1	A	50	98	50	m
1	2	A	60	97	60	w
2	3	A	78	86	45	w
3	4	A	58	98	30	m

- 몇몇 기준(범주형)에 따라 여러 그룹으로 데이터를 분할(splitting)
 - 예) 반별 그룹
- 각 그룹에 독립적으로 함수를 적용(applying)
 - 예) 반별 평균
- 결과물들을 하나의 데이터 구조로 결합(combining)
 - 예) 반별 평균결합



- 그룹 평균 : `means = weather.groupby('month').mean()`
- 그룹 합계 : `sums = weather.groupby('month').sum()`

```
weather = pd.read_csv('https://raw.githubusercontent.com/dongupak/DataSciPy/master/data/csv/weather.csv',  
                      encoding='CP949')  
  
weather['month'] = pd.DatetimeIndex(weather['일시']).month  
means = weather.groupby('month').mean()  
print(means)
```

	평균기온	최대풍속	평균풍속
month			
1	1.598387	8.158065	3.757419
2	2.136396	8.225357	3.946786
3	6.250323	8.871935	4.390291
4	11.064667	9.305017	4.622483
5	16.564194	8.548710	4.219355
6	19.616667	6.945667	3.461000
7	23.328387	7.322581	3.877419
8	24.748710	6.853226	3.596129
9	20.323667	6.896333	3.661667
10	15.383871	7.766774	3.961613
11	9.889667	8.013333	3.930667
12	3.753548	8.045484	3.817097

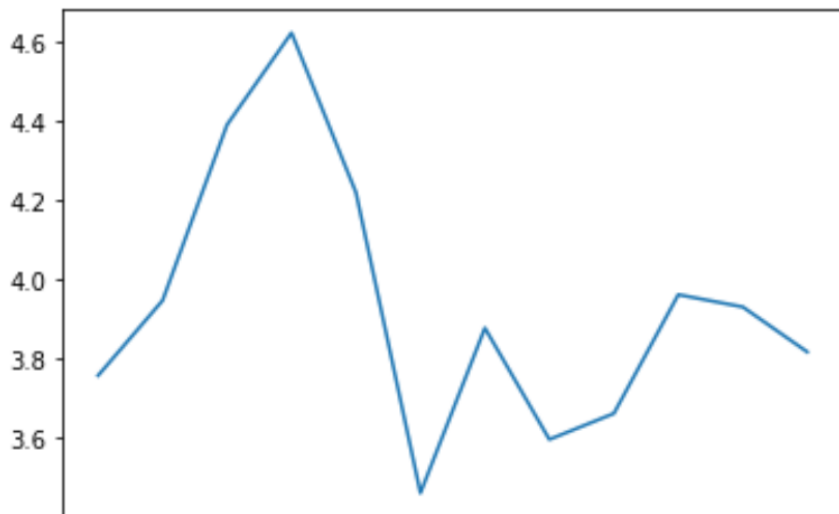
실습. 월 평균 풍속 그래프

```
import pandas as pd
import matplotlib.pyplot as plt
import datetime as dt

weather = pd.read_csv('https://raw.githubusercontent.com/dongupak/DataSciPy/master/data/csv/weather.csv',
                      encoding='CP949')

weather['month'] = pd.DatetimeIndex(weather['일시']).month
means = weather.groupby('month').mean()
means['평균풍속'].plot()

plt.show()
```



- 논리 인덱싱 사용

- 데이터프레임 열벡터에 논리 연산을 적용하여 필터링을 위한 논리 배열 생성

```
import pandas as pd
import datetime as dt

weather = pd.read_csv('weather.csv')

weather['month'] = pd.DatetimeIndex(weather['일시']).month
weather[ weather['최대풍속'] >= 10.0 ]
```

	일시	평균기온	최대풍속	평균풍속	month
9	2010-08-10	25.6	10.2	5.5	8
12	2010-08-13	24.3	10.9	4.6	8
13	2010-08-14	25.0	10.8	4.4	8
14	2010-08-15	24.5	16.9	10.3	8
29	2010-08-30	26.2	10.5	6.2	8
...	...	...	...	...	...

- 판다스 데이터프레임 정렬 메소드
 - `df.sort_values(열이름, ascending=True, inplace=True)`
 - 열이름 → 정렬 기준
 - 정렬 기준으로 [열이름 리스트] 지정 가능
 - `ascending=True` → 오름 차순 정렬
 - `inplace=True` → 현재 데이터프레임 정렬

```
import pandas as pd

df = pd.DataFrame({'name' : ['이길동', '김길동', '홍길동', '박길동'],
                  'class' : ['A', 'A', 'B', 'B'],
                  'score' : [80, 90, 85, 75]}, columns=['name', 'score', 'class'])

print(df)
df.sort_values(['class', 'score'], inplace=True)
print(df)
```

```
name score class
0  이길동    80    A
1  김길동    90    A
2  홍길동    85    B
3  박길동    75    B
name score class
0  이길동    80    A
1  김길동    90    A
3  박길동    75    B
2  홍길동    85    B
```

- Reshaping이라고 함
 - 분석 과정에서 원본 데이터의 구조가 분석 기법에 맞지 않아 행과 열의 위치를 바꾸는 경우
 - 특정 요인에 따라 집계 해서 구조를 변경하는 경우
- 판다스 재구조화 관련 기능

함수	설명
<code>pd.cut(), pd.qcut()</code>	데이터 구간화
<code>pd.get_dummies</code>	원-핫 인코딩
<code>T</code>	데이터 전치
<code>pivot(), pd.pivot_table()</code>	피벗 테이블
<code>melt()</code>	열,행 전환
<code>stack(), unstack()</code>	행,열 인덱스 전환

피벗 테이블(Pivot table)

- 많은 양의 데이터에서 필요한 자료만을 뽑아 새롭게 데이터를 재구성하는 기능
 - 피벗 테이블을 사용하면 사용자가 원하는 대로 데이터를 정렬하고 필터링 가능
- 판다스는 피벗테이블 관련 메소드
 - `df.pivot(index, columns, values)`
 - 첫번째 인수 행 인덱스로 사용할 열 이름, 두번째 인수 열 인덱스로 사용할 열 이름, 세번째 인수 데이터로 사용할 열 이름
 - 데이터 열 중에서 두 개의 열을 각각 행 인덱스, 열 인덱스로 사용하여 데이터를 조회하여 펼쳐 놓은 표(데이터프레임) 반환

```
import pandas as pd

df_1 = pd.DataFrame({'item' : ['ring0', 'ring0', 'ring1', 'ring1'],
                     'type' : ['Gold', 'Silver', 'Gold', 'Bronze'],
                     'price': [20000, 10000, 50000, 30000]})

df_2 = df_1.pivot(index='item', columns='type', values='price')
print(df_2)
```

type	Bronze	Gold	Silver
item			
ring0	NaN	20000.0	10000.0
ring1	30000.0	50000.0	NaN



thank you

본 과제(결과물)는 교육부와 한국연구재단의 재원으로 지원을 받아 수행된
디지털신기술인재양성 혁신공유대학사업의 연구결과입니다.